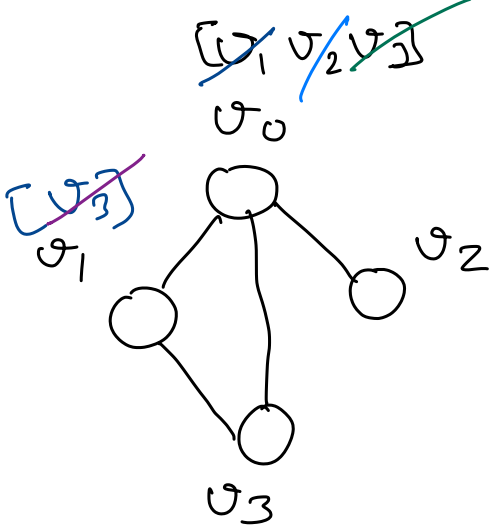
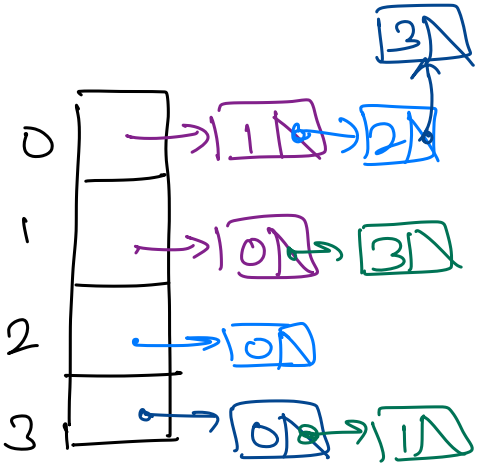
DFS



addEdge(0,1)
addEdge(0,2)
addEdge(0,3)
addEdge(1,3)

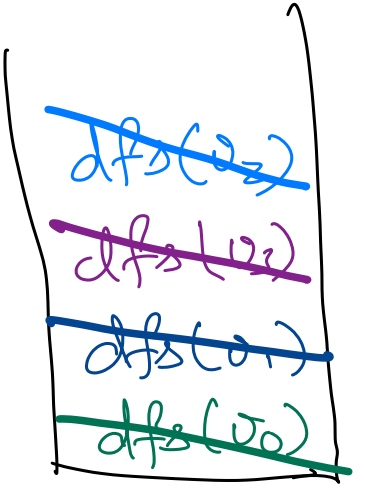


O/p: v_0 v_1 v_3 v_2

0	1	2	3
F T	F T	F T	F T

isVisited

Current → v_0 v_1 v_3 v_1 v_2 v_0

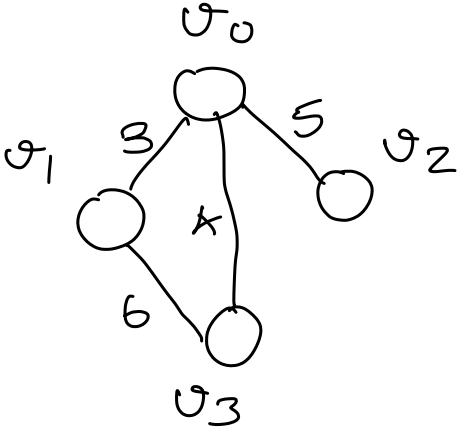


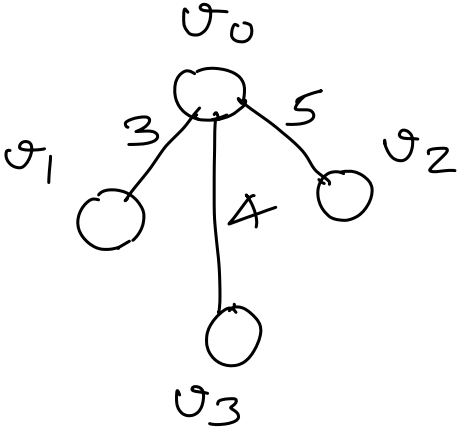
DFS()
- Create isVisited array of size vertexCount.
- Set all elements in isVisited to FALSE.
- DFSHelper(1, isVisited)
- Stop

DFSHelper(startVertex, isVisited)
- if (startVertex is visited) then
  - Stop
- Mark startVertex as visited
- Process startVertex
- For every adjacent vertex, vj, to startVertex that is not visited
  - DFSHelper(vj, isVisited)
- Stop

Spanning Tree

A graph with all vertices that are connected without forming a cycle.

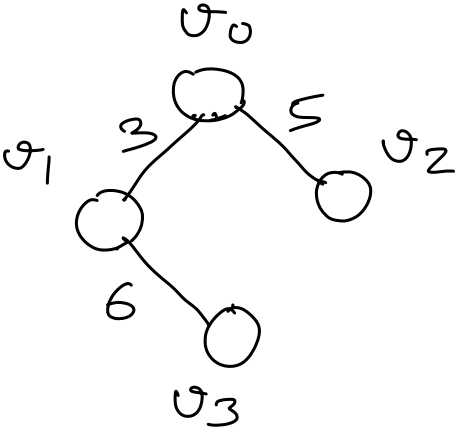




Spanning Tree

weight = 3+5+4

= 12



Spanning Tree

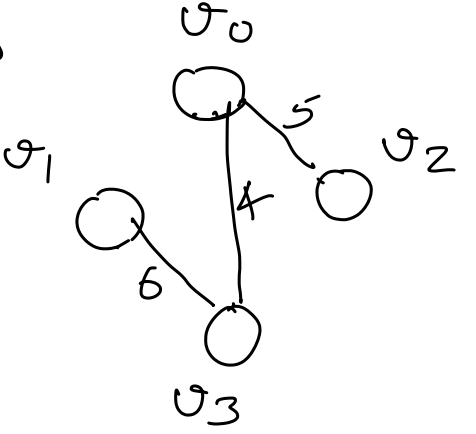
weigh

= 3+5+6

= 14

Minimum Spanning Tree

Spanning Tree



Weight = 5+4+6
= 15

For a weighted graph, minimum spanning tree is a tree in which sum of edge weights is minimum

Brute Force

→ Enumerate all possible solutions.

Greedy Algorithm

At each step we locally optimize the solution.

Kruskal and Prim
Picks edge with minimum weight.

Dijkstra
Instead of picking the edge with the smallest weight, pick the vertex with the smallest distance.

Spanning Tree
A tree that included all vertices of graph, with (|V| - 1) possible edges, such that the graph is connected.

Minimum Spanning Tree
Spanning tree where sum of the weights of edges is minimum.

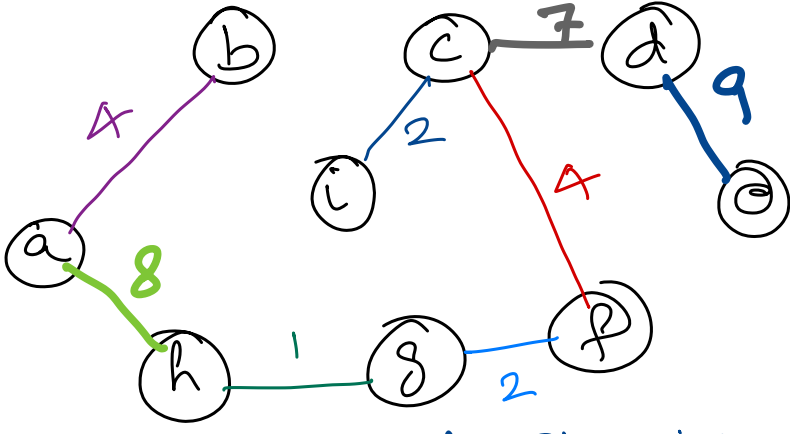
Connected Graph
From any vertex, we can reach any other vertex in the graph.

Exercise: How to check if a graph is connected?

Exercise: If a graph is not connected then how many disconnected parts are there?

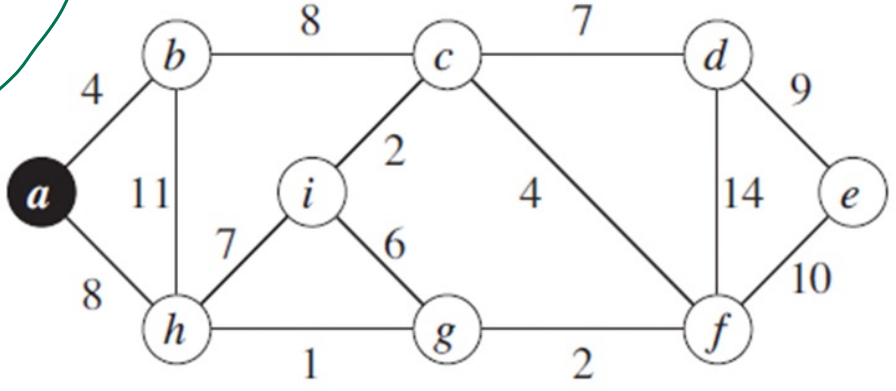
Weight	U	V
1	G	H
2	C	I
2	F	G
4	A	B
4	C	F
6	G	I
7	C	D
7	I	H
8	A	H
8	B	C
9	D	F
10	E	F
11	B	H
14	D	F

$|V| = 9$



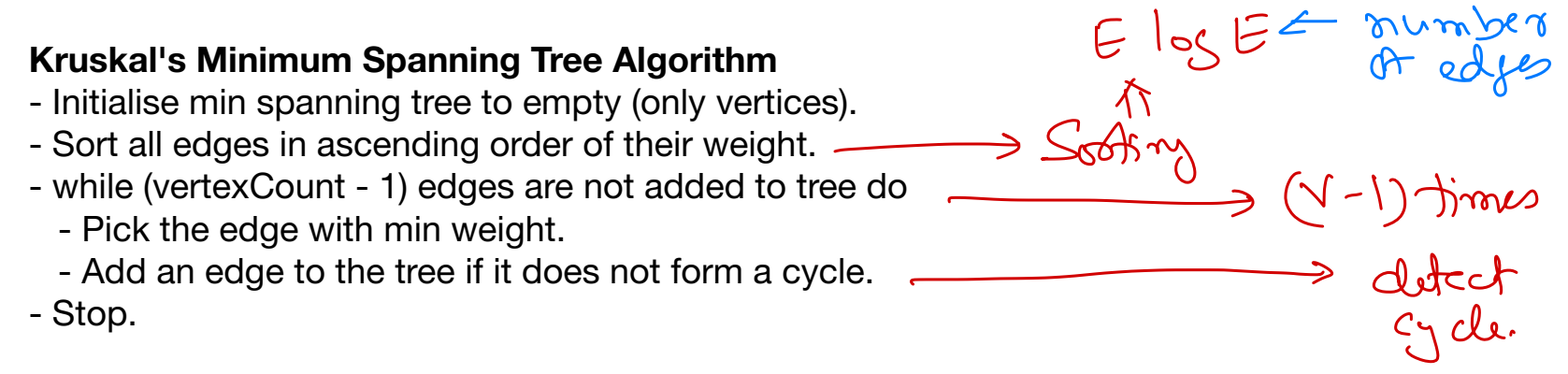
Min Spanning Tree = 37

forms a cycle



Kruskal's Minimum Spanning Tree Algorithm

- Initialise min spanning tree to empty (only vertices).
- Sort all edges in ascending order of their weight.
- while (vertexCount - 1) edges are not added to tree do
  - Pick the edge with min weight.
  - Add an edge to the tree if it does not form a cycle.
- Stop.



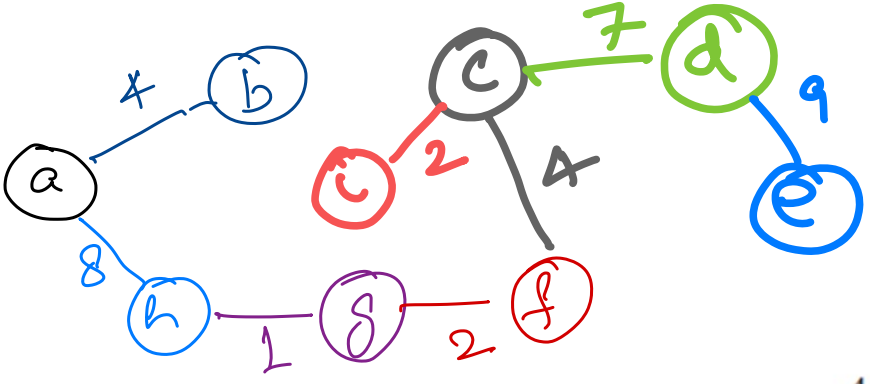
$E = V^2$ ⇒ Max number of edges.

Dense Graph ⇒ we have lots of edges.

Sparse Graph ⇒ we have few number of edges.

Prim's Minimum Spanning Tree Algorithm

- Initialise min spanning tree with any vertex from graph.
- while min spanning tree do not have vertexCount number of vertex
  - Get all edges in the graph that connect the tree 2 newly added
  - Add the edge with min weight to the tree, if no cycle is formed.
- Stop.

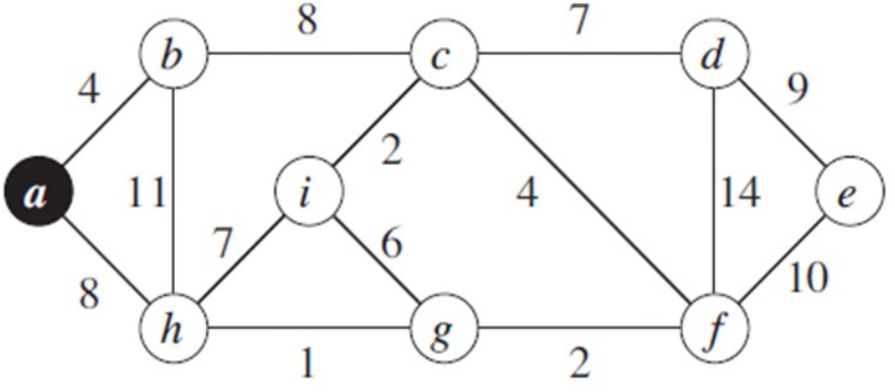


Min Spanning Tree
weight = 37

ab 4
ah 8
bh 11
bc 8
ic 7
hg 1
gi 6
gf 2

fc 4
fd 14
fe 10
ci 2
cd 7
de 9

forma cycle



Dynamic Programming

A problem-solving approach, in which we precompute and store simpler, similar subproblems, in order to build up the solution to a complex problem.

Dynamic programming is used if there is

- **Optimal substructure**
  Problem can be solved using the solution of subproblems.
- **Overlapping subproblems**
  Larger problems are divided into smaller problems.
  And there may be duplicate sub problems.
- **Memoization** - Each unique problem is solved once and its solution is remembered.

Find n^{th} term of fibonacci Series

$$f(n) = f(n-1) + f(n-2)$$

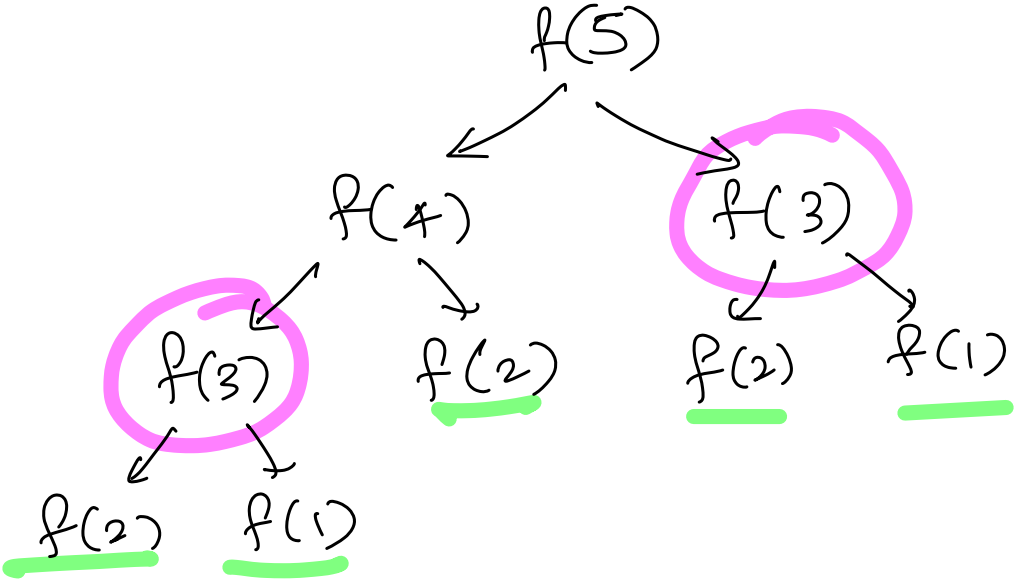
$f(1) \to 1$

$f(2) \to 1$

$$f(n) = \begin{cases} 1, & \text{if } n = 1 \text{ or } n = 2 \\ f(n-1) + f(n-2), & \text{otherwise.} \end{cases}$$

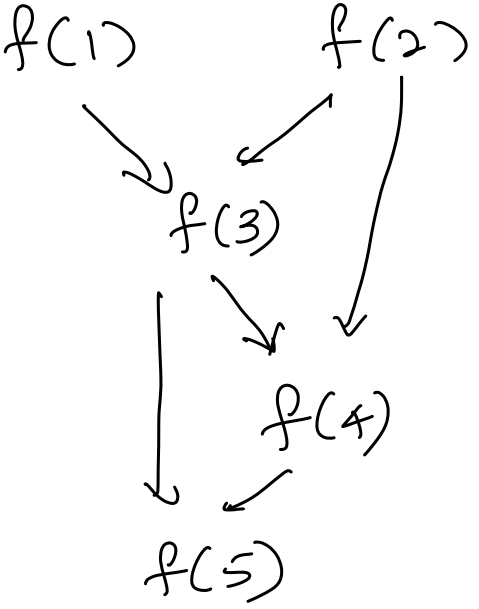
f(5)

Recursion Tree



Top Down : Find solution of larger problem by breaking it into smaller sub problems.

Bottom Up: Build the solution from bottom (Base case) towards larger problem.



Dijkstra's Shortest Path (startVertex, endVertex)

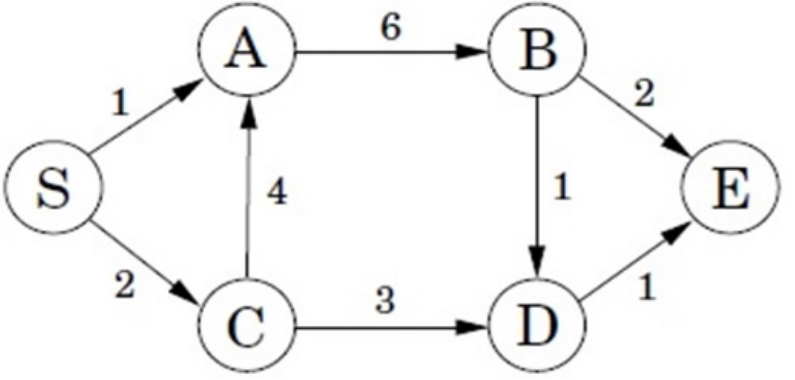
- Set currentDistance for all vertices as infinity.
- Set currentDistance for startVertex as 0.
- Create a list of all vertices in graph, vertexList.
- while vertexList is not empty do
  - Get a vertex, u, from vertexList such that u has smallest distance
  // Update the distance of each vertex v, adjacent to u.
  - distanceToVviaU = currentDistance[u] + weight of edge (u, v)
  - if (currentDistance[v] > distanceToVvisU) then
    - Set currentDistance[v] to distanceToVviaU
    - Set predecessor of v to u.
- Stop

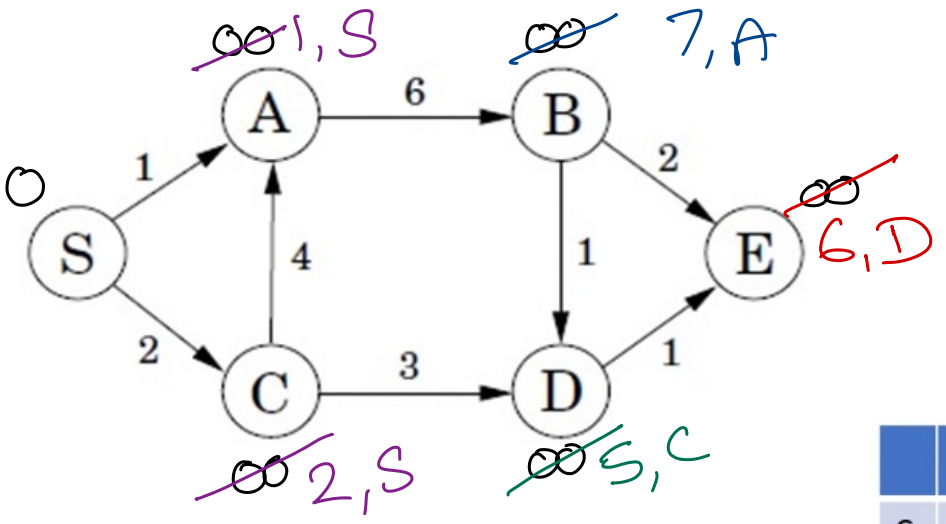
V times

O(V) if linear scan

O(log V) if using Min Heap

Loop





Shortest Path

E ← D ← C ← A ← S

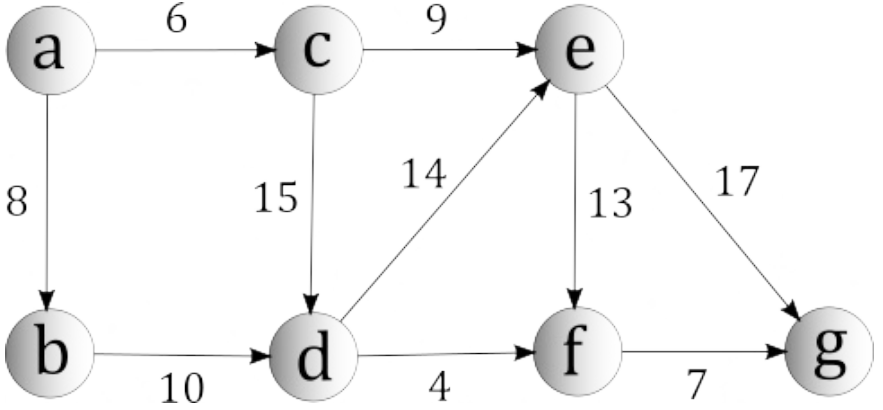
= 6

vertex list

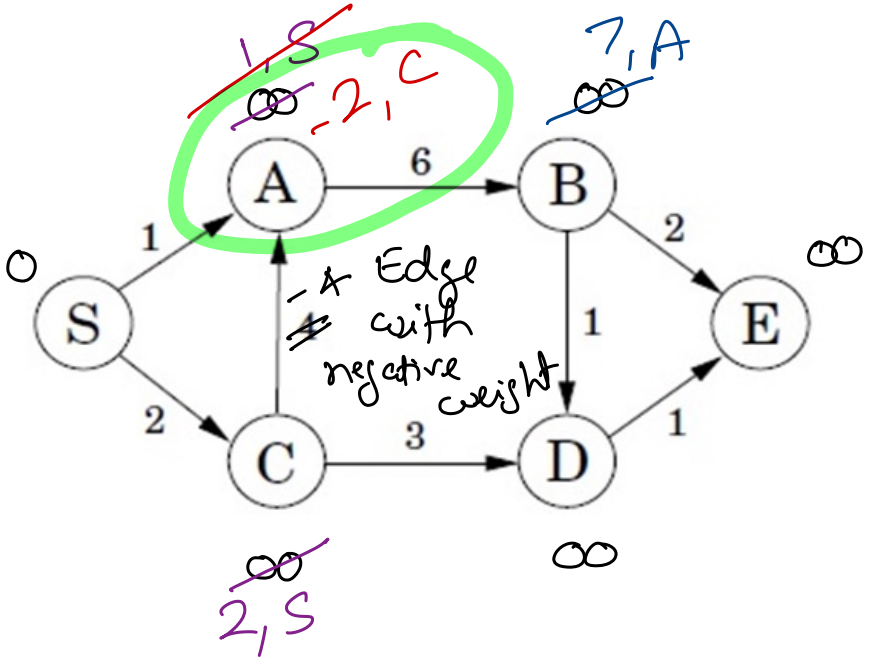
S A B C D E

	Init	S	A	C	D	E	B
S	0						
A	∞	1,S					
B	∞	∞	7,A	7,A	7,A	7,A	
C	∞	2,S	2,S				
D	∞	∞	∞	5,C			
E	∞	∞	∞	∞	6,D		

currentDistance



	Init	a	c	b	e	d	f	g
a	0							
b	∞	8,a	8,a					
c	∞	6,a						
d	∞	∞	21,c	18,b	18,b			
e	∞	∞	15,c	15,c				
f	∞	∞	∞	∞	28,e	22,d		
g	∞	∞	∞	∞	32,e	32,e	29,f	



vertex list

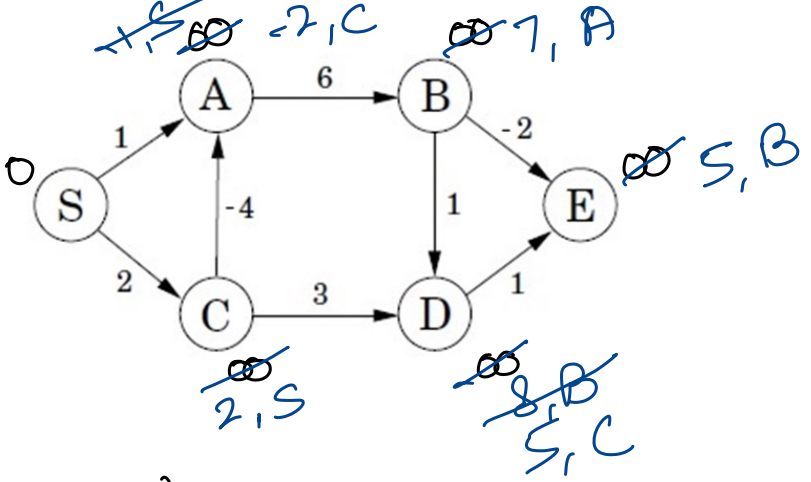
~~S~~ ~~A~~ B ~~C~~ D E

Dijkstra does not work if graph has negative edge weight.

Bellman-Ford Shortest Path Algorithm

Like Dijkstra, it is also a single-source shortest Path algorithm.
It allows edges with negative weight in the graph.

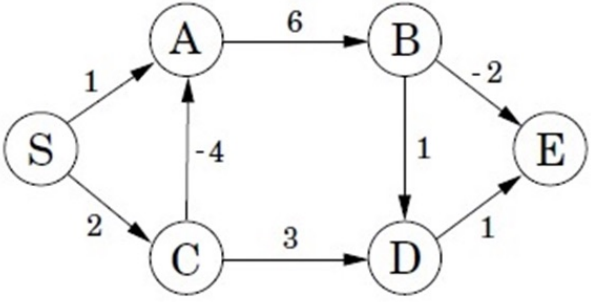
Bellman-Ford Shortest Path Algorithm
- Set the current distance for all vertices as infinity.
- For startVertex, set the current distance to 0.
- Create a list of all edges in the graph.
- For |V| - 1 times do
  - distanceToVviaU = currentDistance[u] + weight of edge (u, v)
  - if (currentDistance[v] > distanceToVviaU) then
    - Set currentDistance[v] to distanceToVviaU
    - Set predecessor of v to u
- Done.



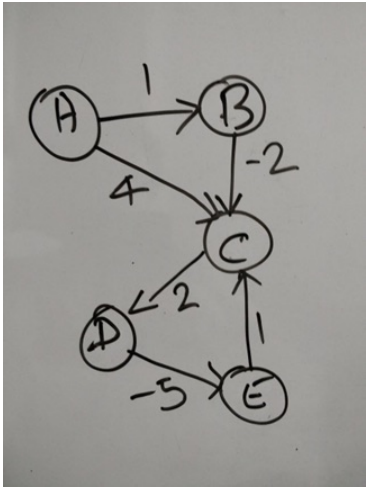
u	v	w
S	A	1
S	C	2
A	B	6
B	D	1
B	E	-2
C	A	-4
C	D	3
D	E	1

	Init	Iter #1	
S	0	0	
A	∞	~~+1, S [S→A]~~	-2, C [C→A]
B	∞	~~7, A [A→B]~~	
C	∞	~~2, S [S→C]~~	
D	∞	~~8, B [B→D]~~	5, C, [C→D]
E	∞	~~5, B [B→E]~~	

currentDistance



	Init	1	2	3	4	5
S	0	0	0	0	0	0
A	∞	-2,C	-2,C	-2,C	-2,C	-2,C
B	∞	7,A	4,A	4,A	4,A	4,A
C	∞	2,S	2,S	2,S	2,S	2,S
D	∞	5,C	5,C	5,C	5,C	5,C
E	∞	5,B	2,B	2,B	2,B	2,B

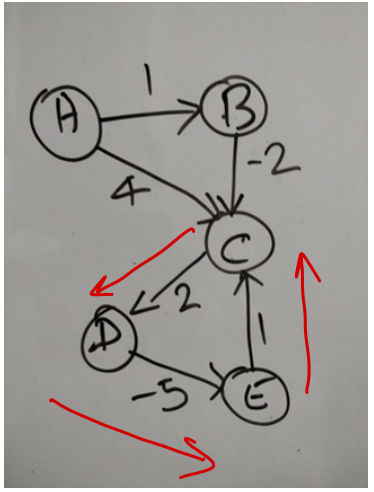


	Init	1	2	3	4
A	0	0	0	0	0
B	∞	1,A	1,A	1,A	1,A
C	∞	-3,E	-5,E	-7,E	-9,E
D	∞	1,C	-1,C	-3,C	-5,C
E	∞	-4,D	-6,D	-8,D	-10,D

Negative weight cycle

⇓

Sum of weight of edges in a cycle is negative.



C → E → D → E

1 + 2 + (-5) = -2

How to detect if graph has negative weight cycle?

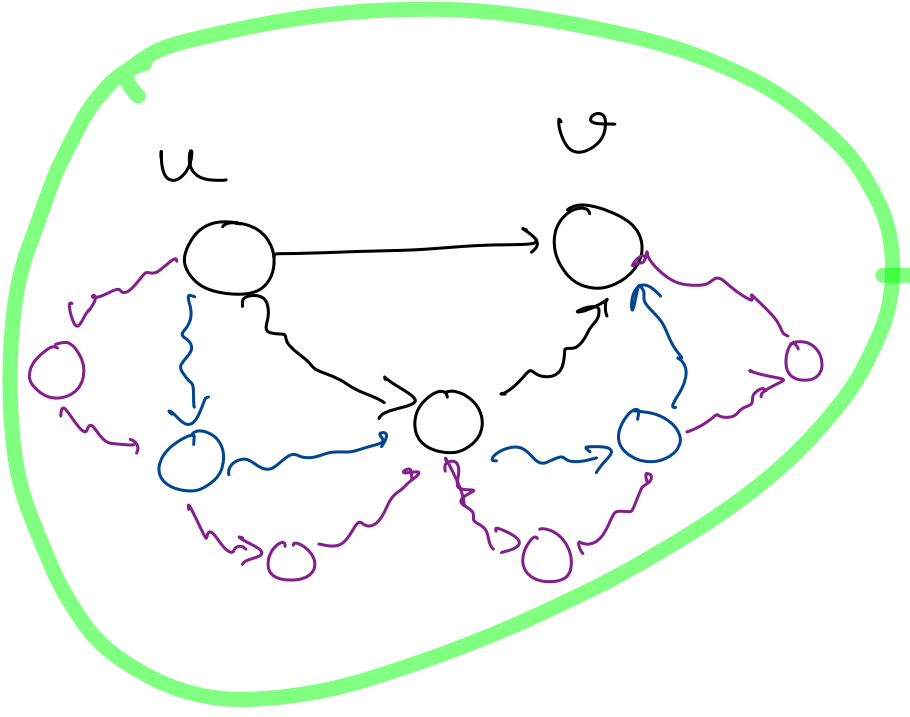
→ Run Bellman-Ford algorithm loop for one more time. → adding a cycle in path.

↓

If currentDistance for any vertex reduces then graph has negative weight cycle.

All Pair Shortest Path

→ Use Bellman Ford by running it for every vertex as a source vertex.



→ 2

(n-2) remaining vertices

possible paths = (n-2)!

$n \times (n-1) \times (n-2)!$

$= n!$

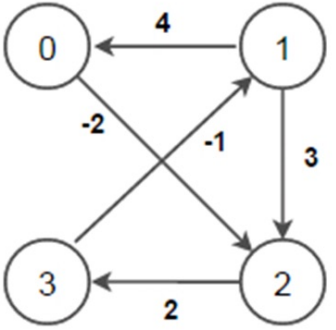
Floyd-Warshall - All-pairs shortest path

Find shortest paths in a weighted graph with positive and negative edge weights (but no negative weight cycles).

Break every possible path between two vertices (u, v) by inserting another vertex k.

Compare the sum of distance of two new paths formed (uk, kv) with the original path (uv) and record shorter path.

Floyd-Warshall - All-pairs shortest path
- Start with adjacency matrix, d,representing the graph and initialize path and
  set non-adjacent vertex distanceto infinity
- For each vertex k from 0 to |V| - 1
  - For each vertex u from 0 to |V| - 1
    - For each vertex v from 0 to |V| - 1
      - if (d[u][v] > (d[u][k] + d[k][v])) then
        - d[u][v]= d[u][k] + d[k][v]
        - path[u][v]= path[k][v]
- Stop



K = 0

u → 1

v → 2

d(u, v) → d(1, 2)

= 3

d(u, k) + d(k, v)

= d(1, 0) + d(0, 2)

= 4 + -2

= 2

Init	0	1	2	3
0	0	∞	-2,0	∞
1	4,1	0	3,1	∞
2	∞	∞	0	2,2
3	∞	-1,3	∞	0

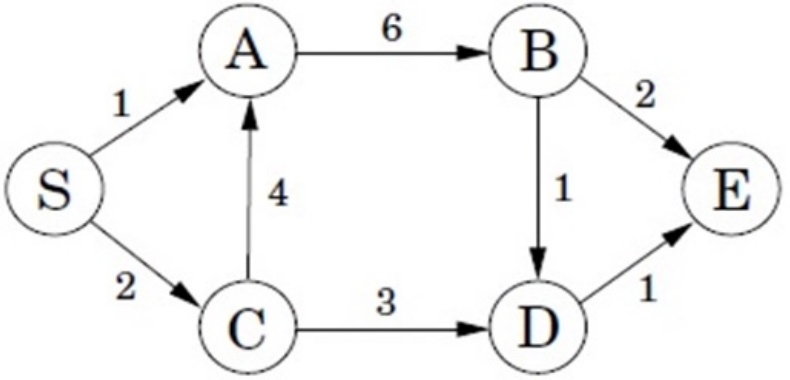
K = 0	0	1	2	3
0	0	∞	-2,0	∞
1	4,1	0	2,0	∞
2	∞	∞	0	2,2
3	∞	-1,3	∞	0

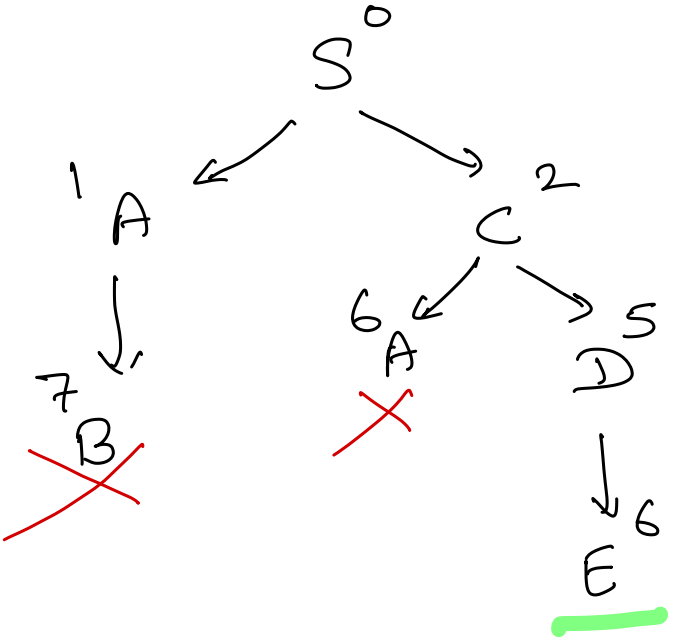
K = 1	0	1	2	3
0	0	∞	-2,0	∞
1	4,1	0	2,0	∞
2	∞	∞	0	2,2
3	3,1	-1,3	1,0	0

K = 2	0	1	2	3
0	0	∞	-2,0	0,2
1	4,1	0	2,0	4,2
2	∞	∞	0	2,2
3	3,1	-1,3	1,0	0

K = 3	0	1	2	3
0	0	-1,3	-2,0	0,2
1	4,1	0	2,0	4,2
2	5,1	1,3	0	2,2
3	3,1	-1,3	1,0	0

Branch and Bound → Optimization Problems
↳ Minimizing.





Union - Find

Union → connect two objects.

Find → Is there a path between two objects.

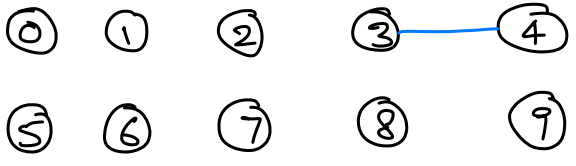
Quick Find

→ Data structure : integer array, ids, of size N.

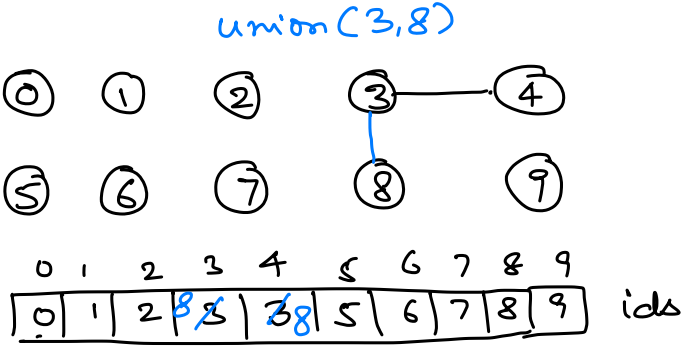
→ find (p, q) : if ids[p] = ids[q] => they are connected.

→ union (p, q) : change all entries with id equal to ids[p] to ids[q].

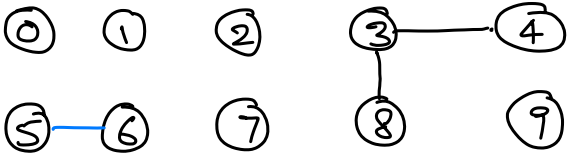
union (4,3)



union (3,8)



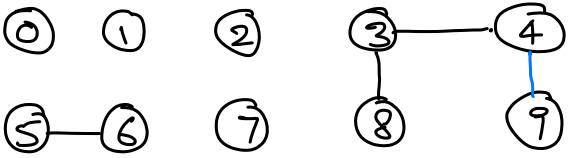
union (6, 5)



0	1	2	3	4	5	6	7	8	9
0	1	2	8	8	5	~~5~~ 6	7	8	9

ids

union (9, 4)

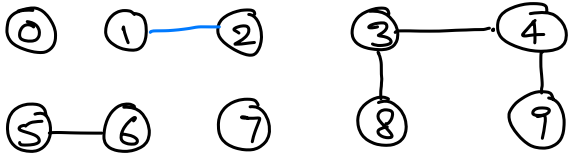


0	1	2	3	4	5	6	7	8	9
0	1	2	8	8	5	5	7	8	~~9~~ 8

ids

union (8, 9) => already connected as ids[8] = ids[9]

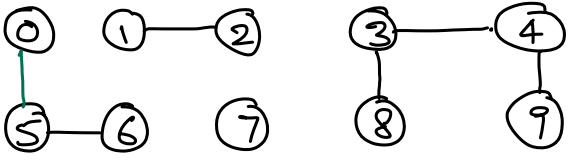
union (2, 1)



0	1	2	3	4	5	6	7	8	9
0	~~1~~ 2	2	8	8	5	5	7	8	8

ids

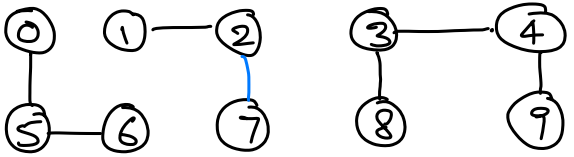
union (5, 0)

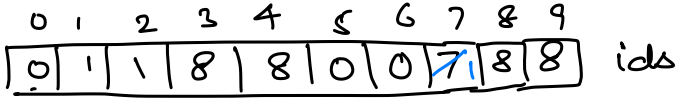


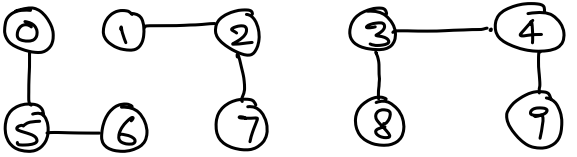
0	1	2	3	4	5	6	7	8	9
0	1	1	8	8	~~5~~ 0	~~5~~ 0	7	8	8

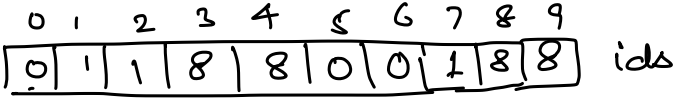
ids

union (7, 2)

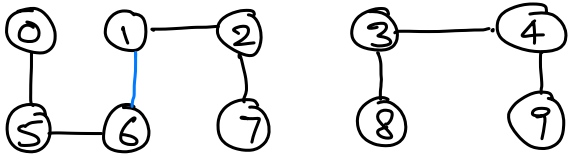


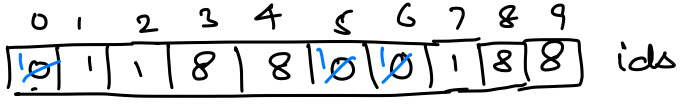


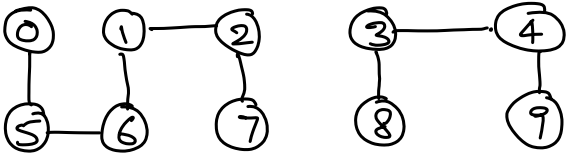


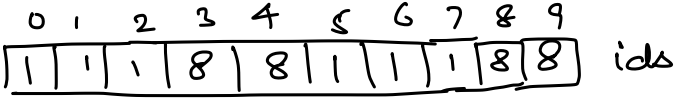


union (6, 1)









```
boolean find ( int p, int q) {
    if ( ids [p] == ids [q])
        return true;

    return false;
}
void union ( int p, int q){
    int pid = ids [p];
    int qid = ids [q];
    if ( pid == qid) return;

    for (int i = 0; i < n; ++i) {
        if (ids [i] == pid) {
            ids [i] = qid;
        }
    }
}
```

← Quick find

O(1)

← O(n)

Not very efficient.